

```

# =====
# GRADIENT BOOSTING IMPLEMENTATION
# =====

# Step 1: Import Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error

# -----
# Step 2: Create Dataset
# -----

data = {
    'Hours_Studied': [1,2,3,4,5,6,7,8,9,10],
    'Marks': [10,20,30,40,50,60,70,80,90,100]
}

df = pd.DataFrame(data)

print(df)

# -----
# Step 3: Split Dataset
# -----

X = df[['Hours_Studied']].values
y = df['Marks'].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# -----
# Step 4: Initialize Parameters
# -----

n_estimators = 5
learning_rate = 0.1

models = []

# -----
# Step 5: Initial Prediction
# -----

F0 = np.mean(y_train)

print("Initial Prediction (F0):", F0)

# Initial predictions
F = np.full(y_train.shape, F0)

# -----
# Step 6: Gradient Boosting Training
# -----

for i in range(n_estimators):

    # Calculate residuals
    residuals = y_train - F

    # Train weak learner
    tree = DecisionTreeRegressor(max_depth=2)

    tree.fit(X_train, residuals)

    # Predict residuals
    pred = tree.predict(X_train)

    # Update predictions
    F = F + learning_rate * pred

    # Save model
    models.append(tree)

print(f"Iteration {i+1} completed")

```

```

print("Iteration 1 completed")

# -----
# Step 7: Predictions on Test Data
# -----

F_test = np.full(y_test.shape, F0)

for tree in models:

    F_test += learning_rate * tree.predict(X_test)

# -----
# Step 8: Evaluate Model
# -----

mse = mean_squared_error(y_test, F_test)

print("Mean Squared Error :", mse)

# -----
# Step 9: Plot Predictions
# -----

plt.scatter(X_test, y_test, label="Actual")
plt.scatter(X_test, F_test, label="Predicted")

plt.xlabel("Hours Studied")
plt.ylabel("Marks")

plt.title("Gradient Boosting Prediction")

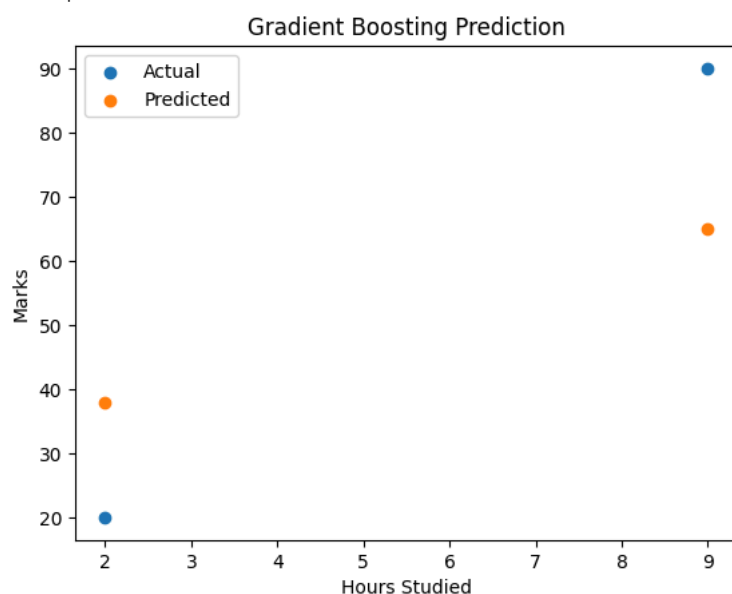
plt.legend()
plt.show()

```

```

Hours_Studied  Marks
0              1    10
1              2    20
2              3    30
3              4    40
4              5    50
5              6    60
6              7    70
7              8    80
8              9    90
9             10   100
Initial Prediction (F0): 55.0
Iteration 1 completed
Iteration 2 completed
Iteration 3 completed
Iteration 4 completed
Iteration 5 completed
Mean Squared Error : 473.86568831864986

```



```
print("# -----# Completed # -----")
```

```
# -----# Completed # -----
```

Start coding or [generate](#) with AI.

